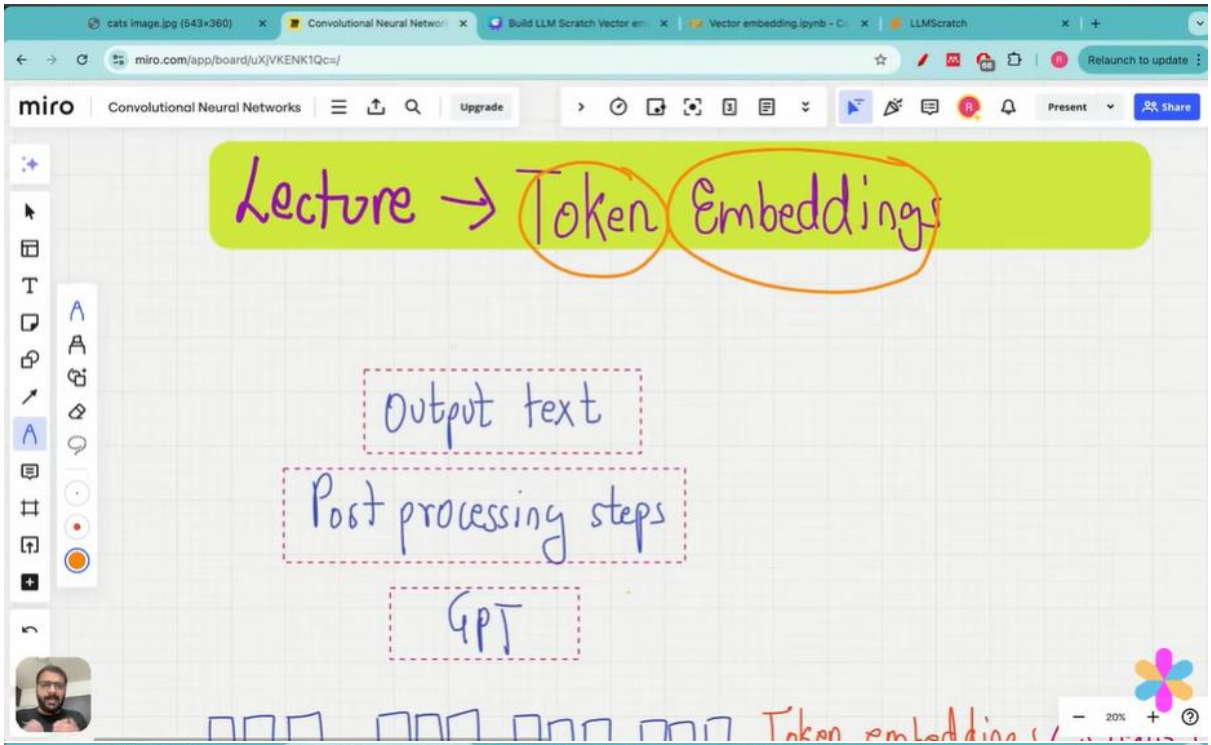
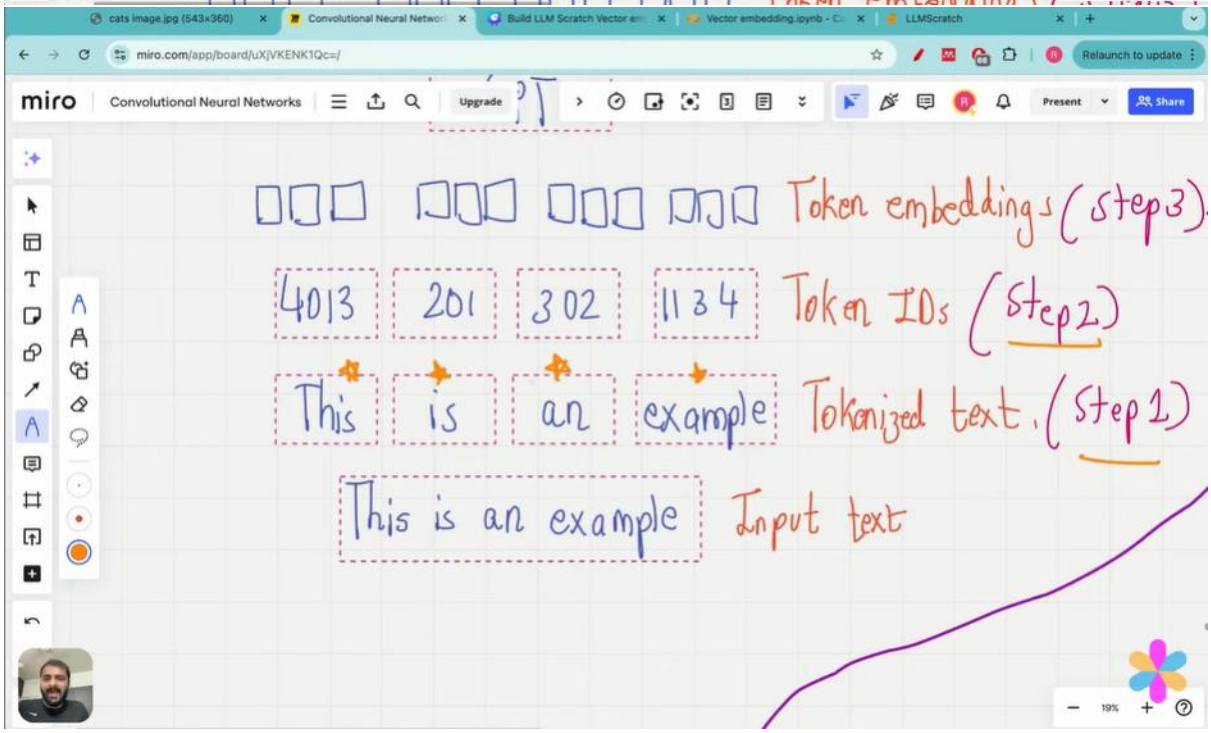
Lec10


miro
Convolutional Neural Networks
Upgrade
Present
Share
Lecture → Token Embeddings
Output text
Post processing steps
GPT


miro
Convolutional Neural Networks
Upgrade
Present
Share
Token embeddings (step3)
4013
201
302
1134
Token IDs (step2)
This
is
an
example
Tokenized text. (step1)
This is an example
Input text

Representing Words Numerically

- Computers need numerical representation of words
- How can we represent words in numbers?

Can we assign random numbers to each word?

"cat"	→	34
"book"	→	2.9
"tablet"	→	-20
"kitten"	→	-13



The Problem With Using Random Numbers

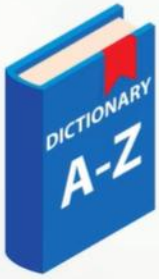
"cat"	→	34
"book"	→	2.9
"tablet"	→	-20
"kitten"	→	-13

"cat" and "kitten" are semantically related.
However the associated numbers 34 and -13 cannot capture this relation.



What About One-Hot Encoding?

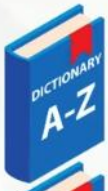
1) Create a dictionary of words
2) Assign sequential one-hot encoding to each word

"dog" → 

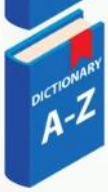
→ [0,0,0,1,0,0,0,0,0,0,0,0,0,0,.....]



The Problem With One-Hot Encoding

"dog" → 

→ [0,0,0,1,0,0,0,0,0,0,0,0,0,0,.....]

"puppy" → 

→ [0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,.....]

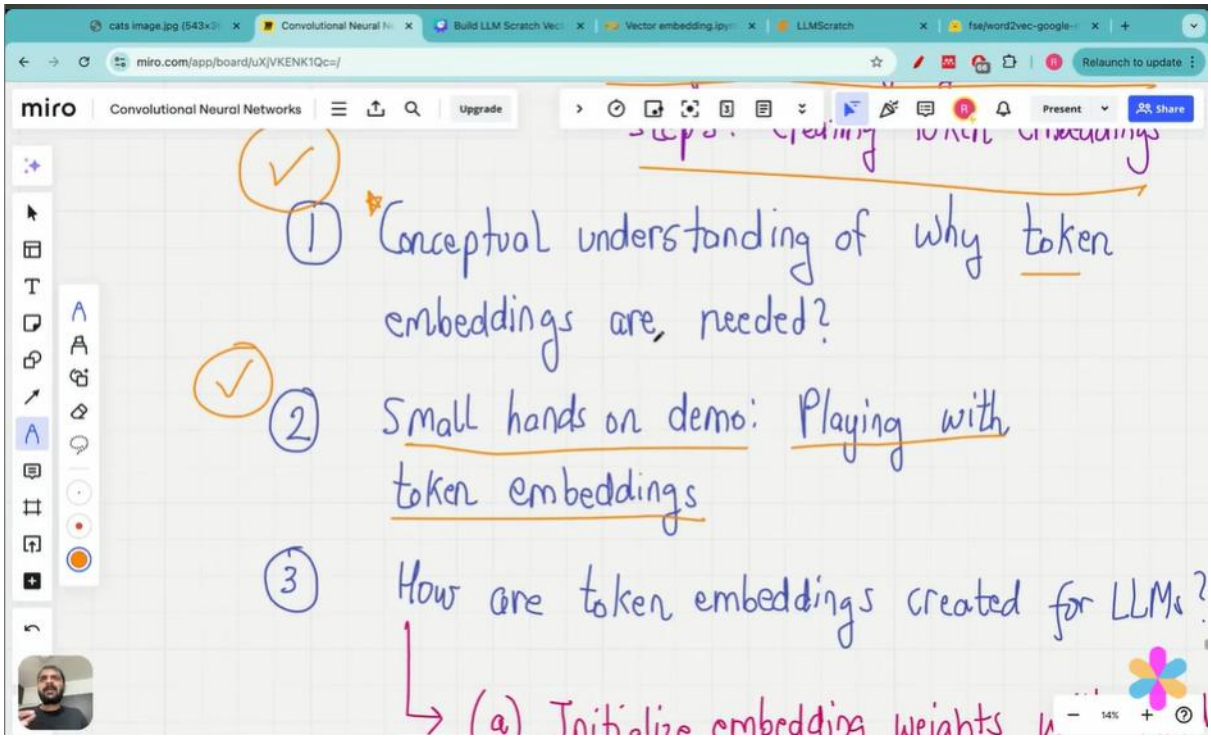
One-hot encoding also fails to capture semantic relationship

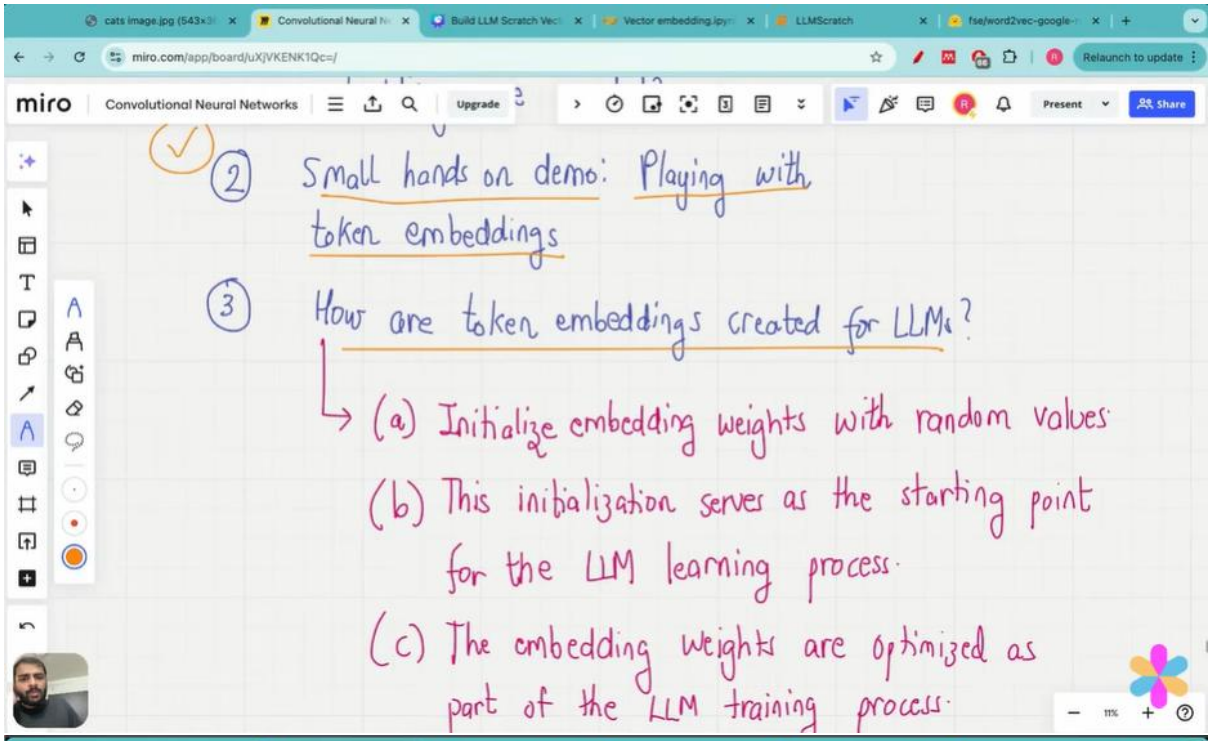


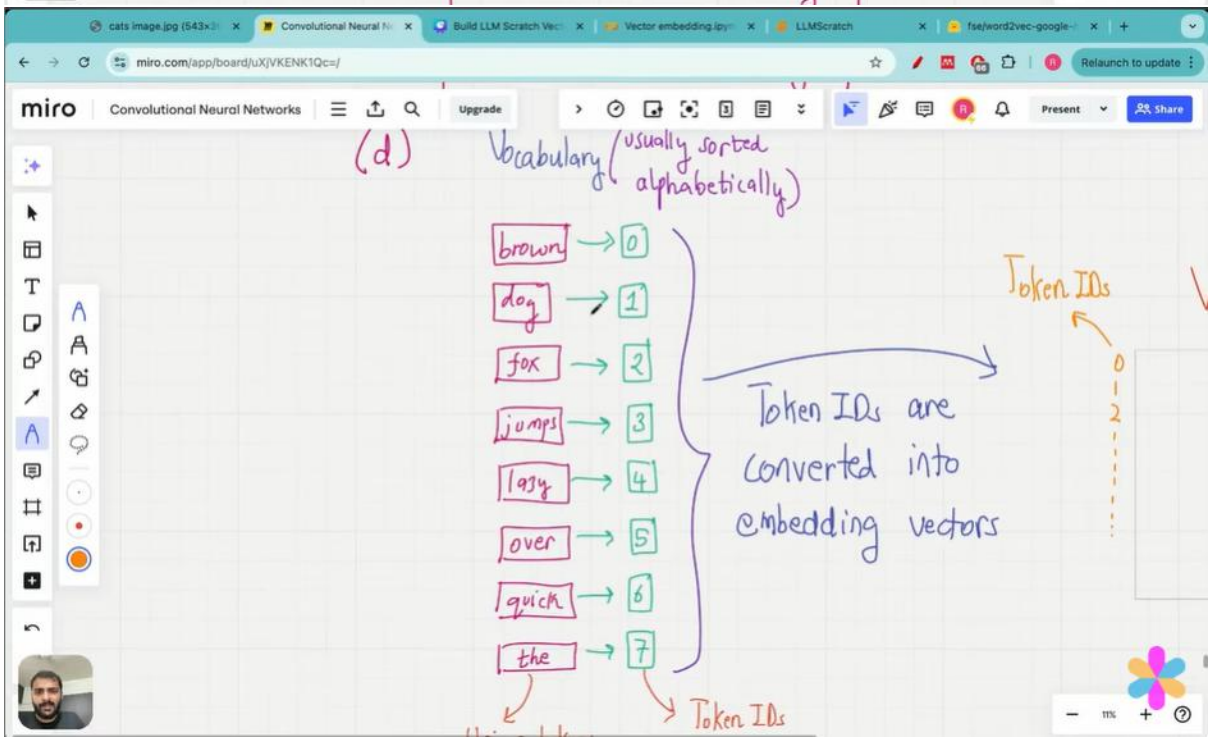
Semantically Similar Words Should Have Similar Vectors

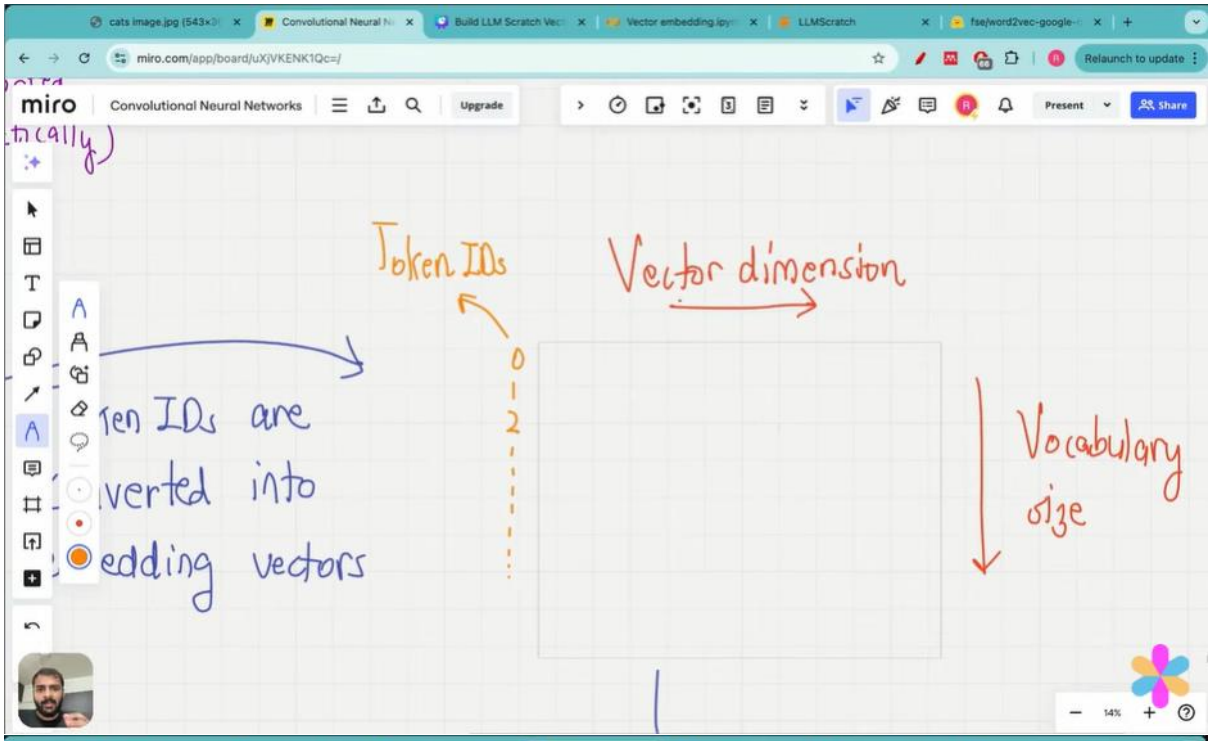
	"dog"	"cat"	"apple"	"banana"
has_a_tail	23	31	1	2
is_eatable	2	3	22	38
has_4_legs	19	21	0	0
makes_sound	12	18	0.5	0.2
is_a_pet	35	31	5	7

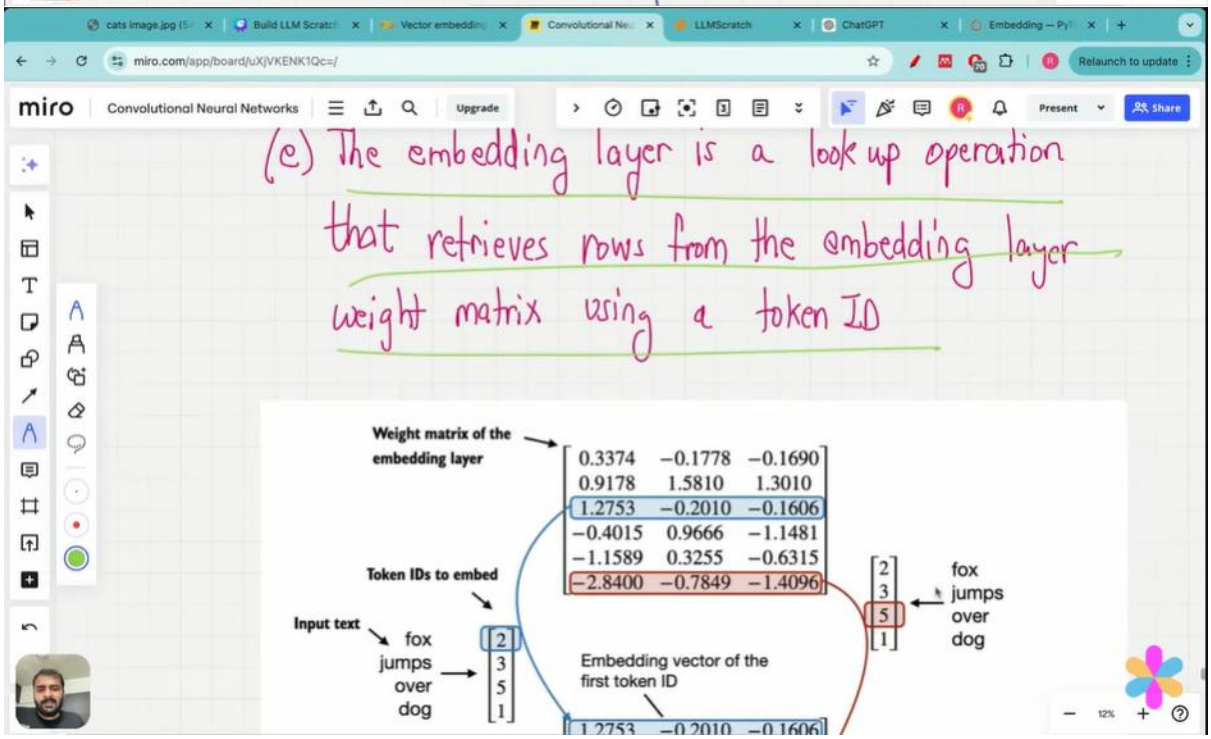


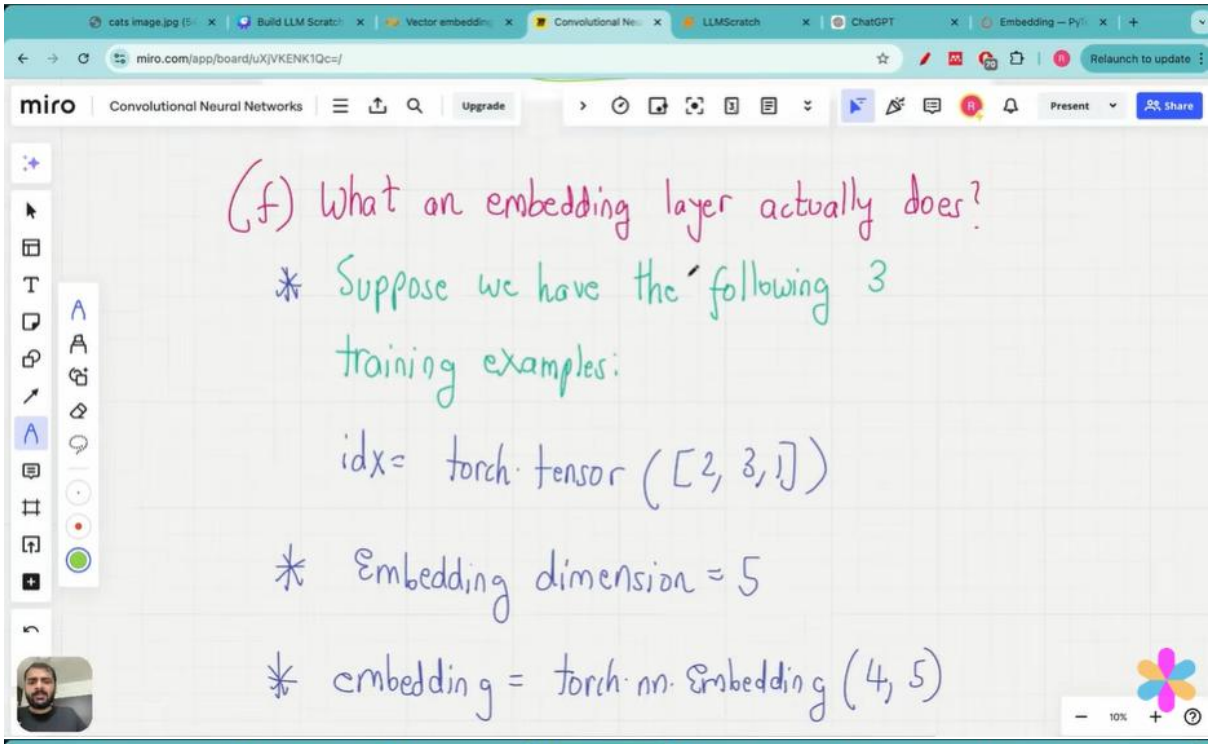


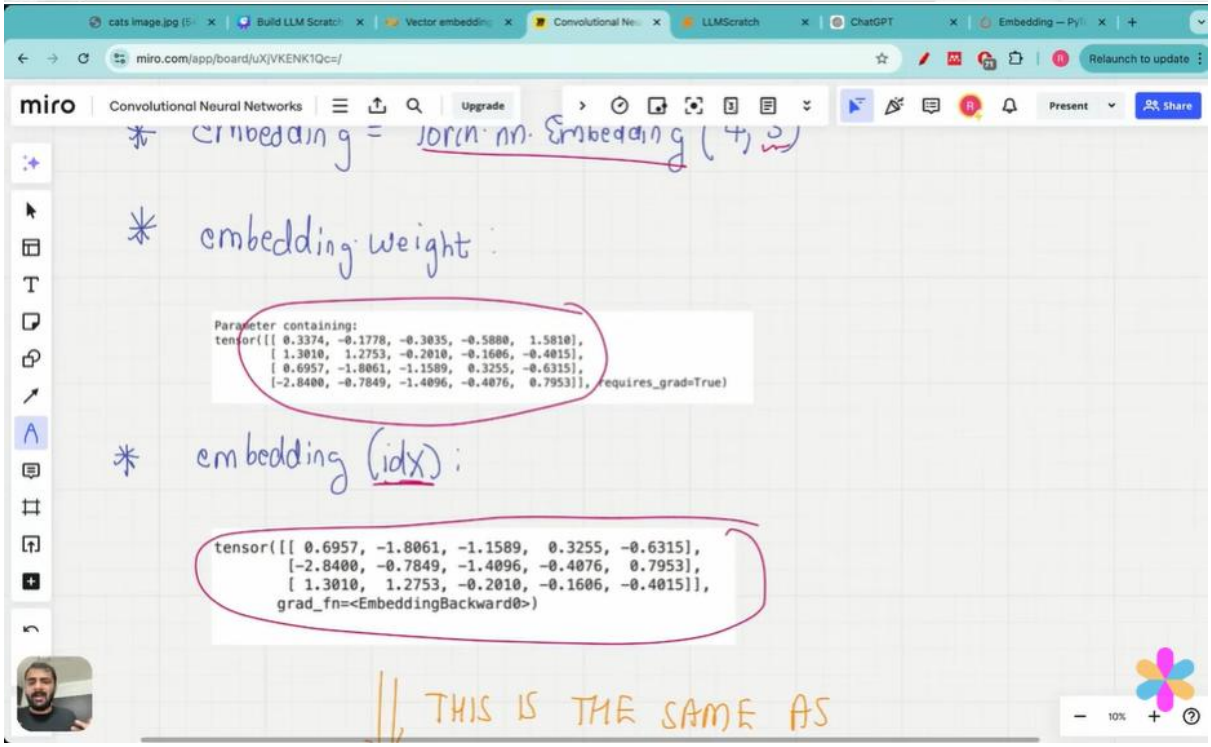

miro
Convolutional Neural Networks
Upgrade
Present
Share
2 Small hands on demo: Playing with token embeddings
3 How are token embeddings created for LLMs?
(a) Initialize embedding weights with random values.
(b) This initialization serves as the starting point for the LLM learning process.
(c) The embedding weights are optimized as part of the LLM training process.

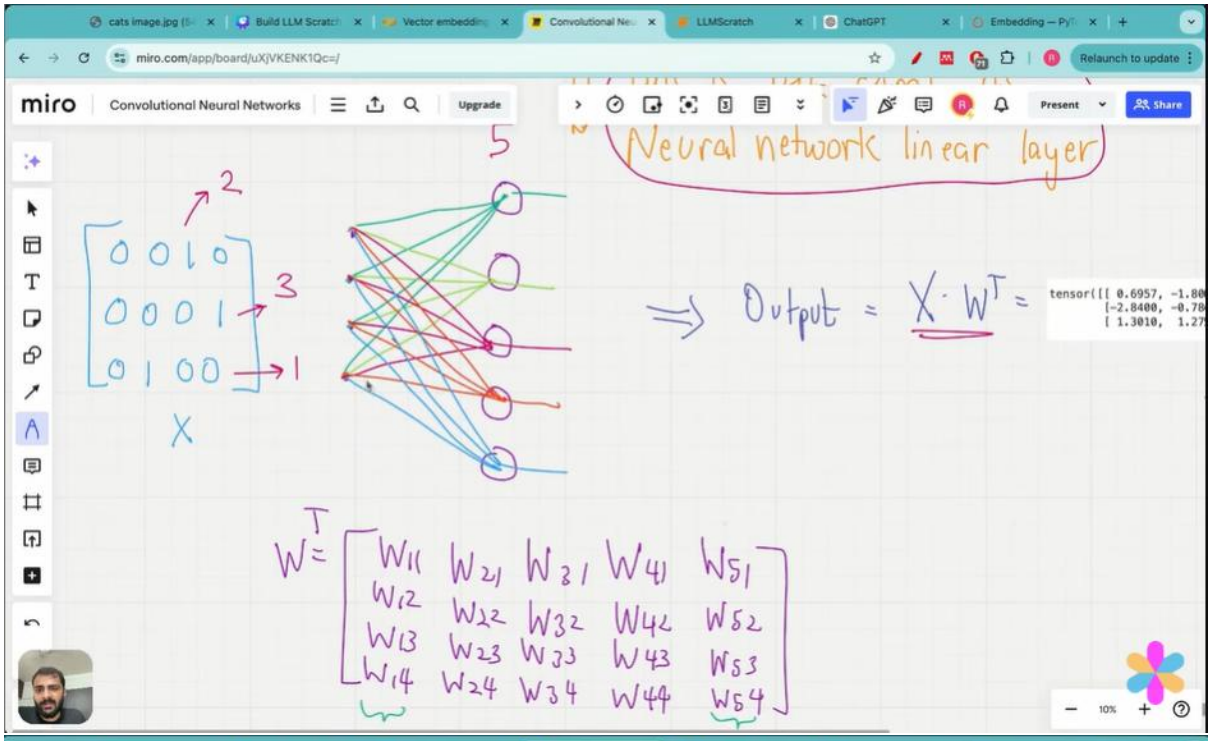

miro
Convolutional Neural Networks
Upgrade
Present
Share
(d) Vocabulary (usually sorted alphabetically)
brown → 0
dog → 1
fox → 2
jumps → 3
lazy → 4
over → 5
quick → 6
the → 7
Token IDs are converted into embedding vectors
Token IDs
0
1
2
Token IDs

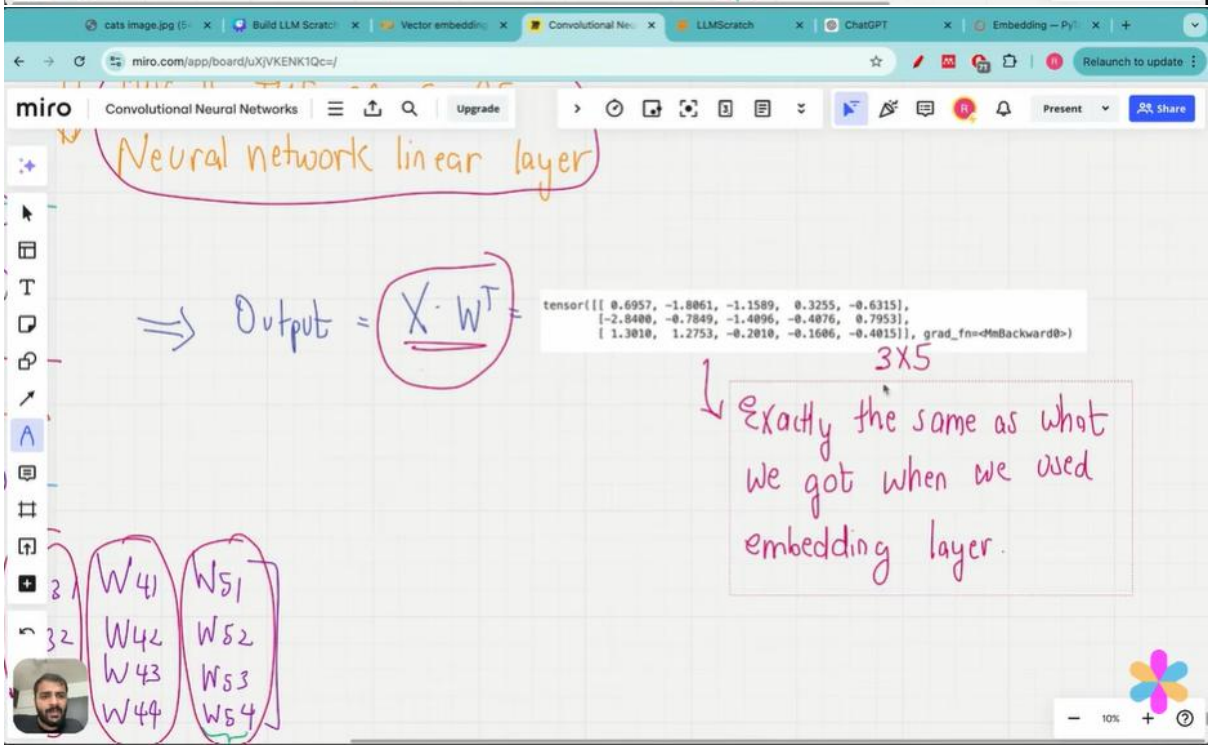

miro
Convolutional Neural Networks
Upgrade
Present
Share
Token IDs
Vector dimension
0
1
2
ren IDs are
verted into
edding vectors
Vocabulary
size

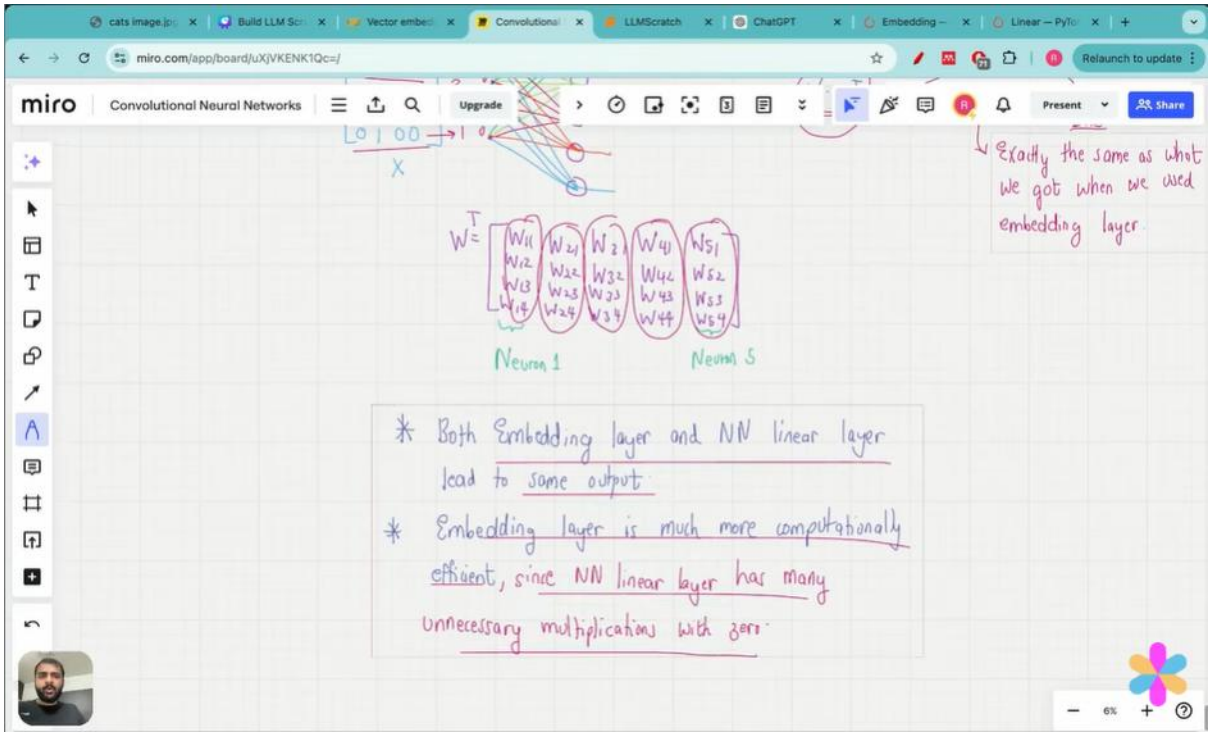

miro
Convolutional Neural Networks
Upgrade
Present
Share
(c) The embedding layer is a look up operation
that retrieves rows from the embedding layer
weight matrix using a token ID
Weight matrix of the
embedding layer
0.3374 −0.1778 −0.1690
0.9178 1.5810 1.3010
1.2753 −0.2010 −0.1606
−0.4015 0.9666 −1.1481
−1.1589 0.3255 −0.6315
−2.8400 −0.7849 −1.4096
Token IDs to embed
Input text
fox
jumps
over
dog
2
3
5
1
Embedding vector of the
first token ID
fox
jumps
over
dog


miro
Convolutional Neural Networks
Upgrade
Present
Share
(f) What an embedding layer actually does?
* Suppose we have the following 3 training examples:
idx = torch.tensor([2, 3, 1])
* Embedding dimension = 5
* embedding = torch.nn.Embedding(4, 5)


miro
Convolutional Neural Networks
Upgrade
Present
Share
* embedding = torch.nn.Embedding(4, 5)
* embedding.weight:
Parameter containing:
tensor([[0.3374, -0.1778, -0.3035, -0.5880, 1.5810],
[1.3010, 1.2753, -0.2010, -0.1606, -0.4015],
[0.6957, -1.8061, -1.1589, 0.3255, -0.6315],
[-2.8400, -0.7849, -1.4096, -0.4076, 0.7953]], requires_grad=True)
* embedding(idx):
tensor([[0.6957, -1.8061, -1.1589, 0.3255, -0.6315],
[-2.8400, -0.7849, -1.4096, -0.4076, 0.7953],
[1.3010, 1.2753, -0.2010, -0.1606, -0.4015]],
grad_fn=<EmbeddingBackward0>)
|| THIS IS THE SAME AS


miro
Convolutional Neural Networks
Upgrade
Present
Share
Neural network linear layer
5
2
3
1
X
Output = X · W^T =
W^T =
W11 W21 W31 W41 W51
W12 W22 W32 W42 W52
W13 W23 W33 W43 W53
W14 W24 W34 W44 W54


miro
Convolutional Neural Networks
Upgrade
Present
Share
Neural network linear layer
Output = X · W^T =
tensor([[0.6957, -1.8061, -1.1589, 0.3255, -0.6315],
[-2.8400, -0.7849, -1.4096, -0.4076, 0.7953],
[1.3010, 1.2753, -0.2010, -0.1606, -0.4015]], grad_fn=<MmBackward0>)
3X5
Exactly the same as what we got when we used embedding layer.
W41 W51
W42 W52
W43 W53
W44 W54


miro
Convolutional Neural Networks
Upgrade
Present
Share
Relaunch to update
miro.com/app/board/uXjVKENK1Qc=/
X
W^T =
Neuron 1
Neuron 5
Exactly the same as what we got when we used embedding layer.
* Both Embedding layer and NN linear layer lead to same output
* Embedding layer is much more computationally efficient, since NN linear layer has many unnecessary multiplications with zero
6%